第1回: 開発環境の準備とインターネットの仕組み

Webアプリケーション基礎 2026

今日のゴール

開発環境が動き、Webの仕組みを体感する

今日の流れ

前半

- コース全体像とWebアプリとは
- GitとGitHubとは
- GitHub Codespacesとは

後半

- インターネットの仕組み
- クライアントとサーバー
- HTTPの詳細

1. コース全体像とWebアプリとは

このコースで作るもの: TODOアプリ

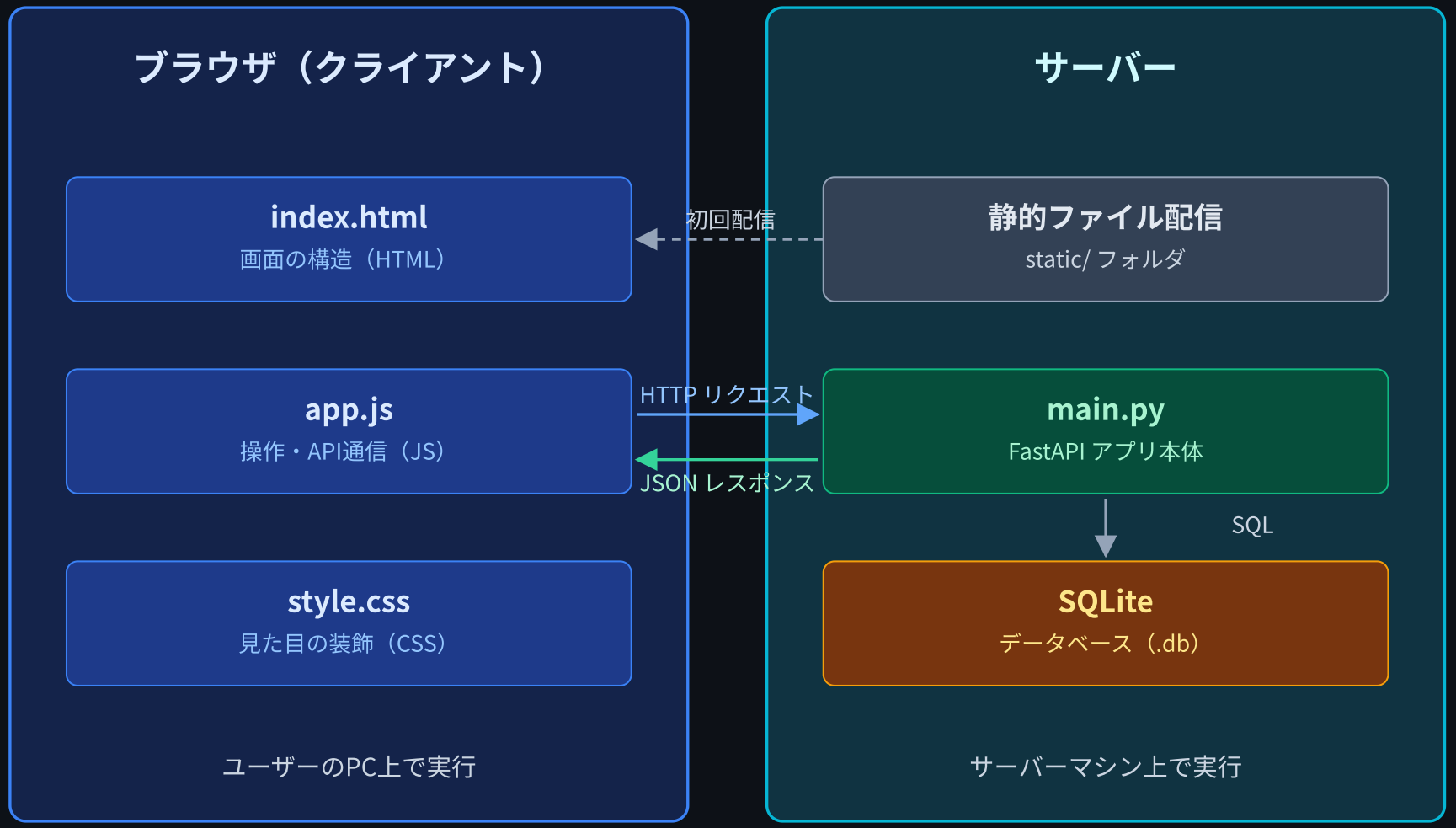
このコースでは全回を通じて **TODOアプリ** を一から作り上げます。

TODOアプリの機能:

- タスクの追加・表示
- タスクの完了/未完了の切り替え
- タスクの削除
- データの永続化（ページを閉じてもデータが残る）

最終的に**インターネット上に公開**して、誰でもアクセスできる状態にします。

TODOアプリの全体アーキテクチャ



Webアプリとは何か？

Webアプリケーション = ブラウザ上で動くアプリケーション

身近な例:
Gmail
YouTube
Twitter / X
Amazon
Google Docs

特徴: **インストール不要**。ブラウザとインターネットがあれば使える。

「ブラウザで動くコード」と「サーバーで動くコード」

Webアプリは **2つの場所** でコードが動いています。

	ブラウザ（フロントエンド）	サーバー（バックエンド）
場所	ユーザーのPC	インターネット上のコンピュータ
言語	HTML, CSS, JavaScript	Python, Ruby, Node.js 等
役割	画面の表示、ユーザー操作	データの保存、処理、配信
見えるか	誰でも見える（開発者ツール）	見えない（サーバー内部）

このコースでは **両方** を学びます。

実習1: 完成版TODOアプリを触ってみよう

手順

1. 講師が共有するURLにブラウザでアクセスする
2. TODOアプリでタスクを追加・完了・削除してみる
3. ブラウザの **開発者ツール** を開く
   - Windows: `F12` または `Ctrl + Shift + I`
   - Mac: `Cmd + Option + I`
4. **Networkタブ** を選択する
5. TODOを追加して、通信が発生することを確認する

注目ポイント: ボタンを押すたびに通信が発生している = ブラウザとサーバーがやりとりしている

2. GitとGitHubとは

バージョン管理の必要性

こんな経験はありませんか？

```
レポート_最終版.docx
レポート_最終版2.docx
レポート_最終版_修正.docx
レポート_本当の最終版.docx
レポート_提出用_確定.docx
```

問題点:

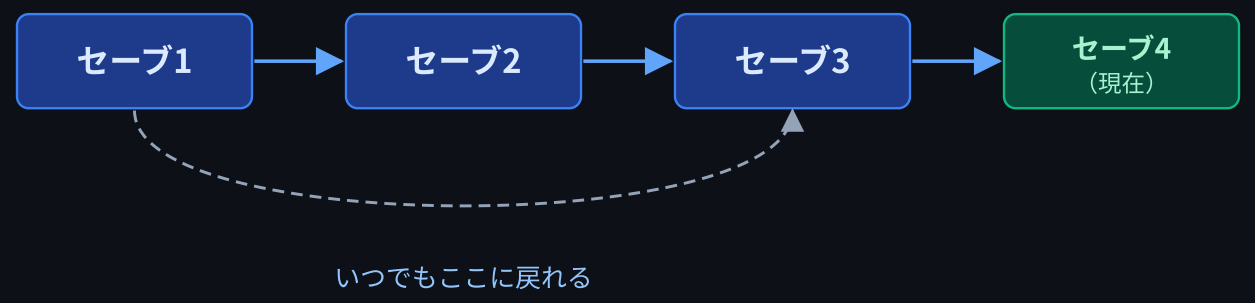
- どれが最新かわからない
- いつ何を変えたか不明
- 前の状態に戻せない
- 複数人で作業すると混乱する

Git はこれらの問題をすべて解決します。

Gitとは — 「セーブポイント」の仕組み

Git = ファイルの変更履歴を記録・管理するツール

ゲームのセーブに例えると:

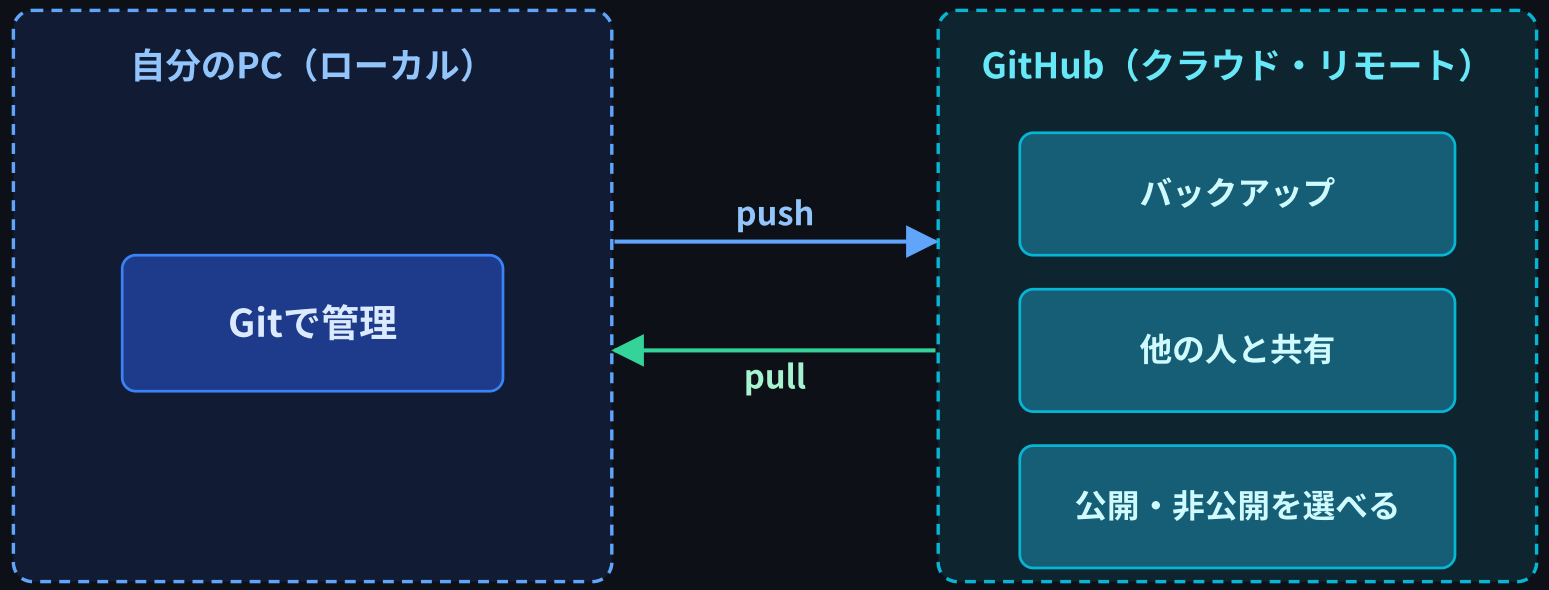


Gitの用語:

ゲーム	Git	意味
セーブする	コミット（commit）	現在の状態を記録する
セーブデータ	リポジトリ（repository）	全履歴を含むプロジェクト全体
セーブポイントに戻る	チェックアウト（checkout）	過去の状態に戻す

GitHubとは

GitHub = Gitのデータをインターネット上に保存・共有するサービス



- **Git**: ツールそのもの（ローカルで動く）
- **GitHub**: Gitのデータを置くWebサービス（クラウド）

GitHubの便利な機能

機能	説明
リポジトリ	プロジェクトの保管場所
Fork	他人のリポジトリを自分のアカウントにコピー
Pull Request	コードの変更を提案・レビューする仕組み
Issues	バグ報告や機能リクエストを管理
GitHub Pages	Webページを無料で公開
Codespaces	ブラウザ上で開発環境を使える

このコースでは **Fork**, **Codespaces**, **コミット** を主に使います。

実習2: GitHubアカウントを作成し、リポジトリをForkする

手順

1. https://github.com にアクセス
2. 「Sign up」からアカウントを作成する
    - 既にアカウントがある人はログインする
3. 講師が共有するテンプレートリポジトリのURLにアクセス
4. 画面右上の **「Fork」** ボタンをクリック
5. 「Create fork」をクリック
6. 自分のアカウントにリポジトリがコピーされたことを確認
7. コピーされたリポジトリのURLをフォームで提出する

確認ポイント: URLが `github.com/自分のユーザー名/リポジトリ名` になっていればOK

3. GitHub Codespacesとは

開発環境とは

プログラミングをするために必要なもの:

必要なもの	説明	従来の方法
テキストエディタ	コードを書くソフト	VS Codeをインストール
プログラミング言語	Python等	公式サイトからインストール
ターミナル	コマンドを実行する画面	OS標準のものを使う
Git	バージョン管理ツール	インストール & 設定

従来はこれらを**1つずつ自分のPCにインストール**する必要がありました。
環境構築でつまずいて、プログラミングに到達できないことも...

GitHub Codespaces — クラウド上の開発環境

GitHub Codespaces = GitHubが提供する**クラウド開発環境**サービス

仕組み:

- **VS Code・Python・Git など開発に必要なツールが全部入った状態** で立ち上がる
- 自分のPCからは **ブラウザ** で接続して操作する（手元のVS Codeから繋ぐこともできる）

メリット:

- **インストール不要** — 自分のPCにPythonやGitを入れる必要がない
- **どのPCでも同じ環境** — Windows / Mac / Chromebook、結果は全員同じ
- **すぐ始められる** — リポジトリの「Code」ボタンから数十秒で起動

GitHub Codespaces — 設定ファイルと利用枠

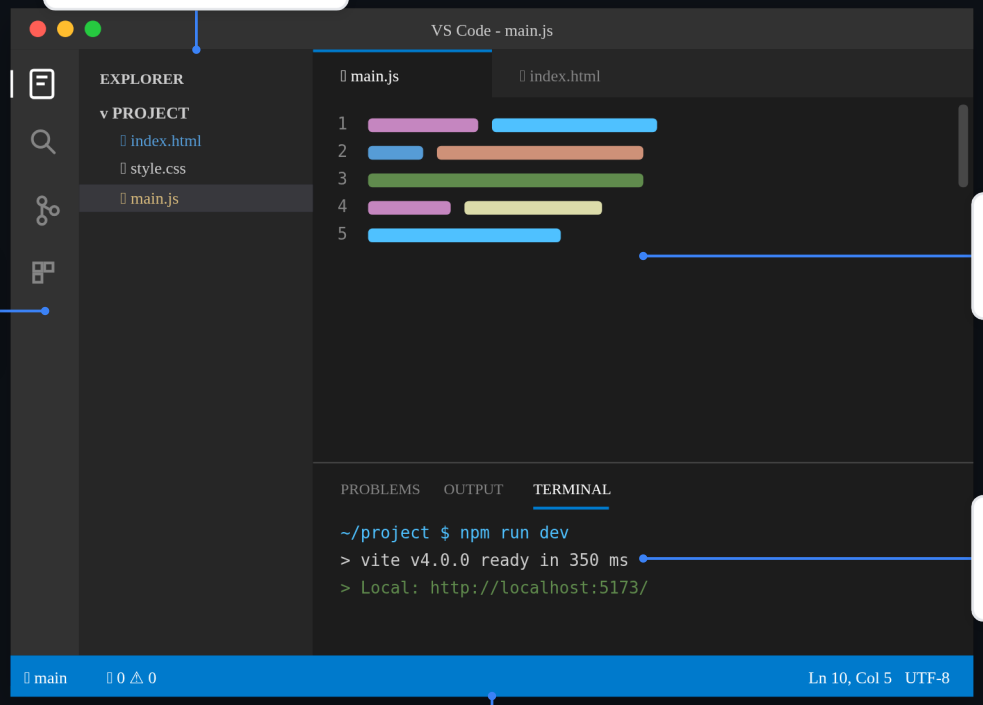
リポジトリに `.devcontainer/devcontainer.json` を置くと、**環境の中身を設定ファイルとして共有** できます。
このリポジトリにも入っていて、Python や VS Code 拡張機能があらかじめ指定されています。

無料枠（個人の無料GitHubアカウント）:

- 月 **120 コア時間** のコンピュート（2コアの標準マシンで約60時間）／ 月 **15 GB** ストレージ
- 毎月1日にリセット。使わないときは**自動停止**するので、こまめに止めれば十分足ります

VS Codeの基本画面

VS Codeの画面は大きく4つのエリアに分かれています。



サイドバー
プロジェクトのファイルツリーや
検索結果などを表示する

アクティビティバー
主な機能(エクスプローラー、
検索など)を切り替える

エディター
コードを実際に編集する領域。
上部のタブでファイルを切り替え。

パネル
ターミナルでのコマンド実行や
エラー出力などを表示する

ステータスバー
Gitのブランチやエラーの数、
カーソル位置などの状態を表示

ターミナルとは

ターミナル（端末）= 文字でコンピュータに命令を出す画面

普段はマウスでクリックして操作しますが、ターミナルでは **文字（コマンド）** で操作します。

```
# ファイルの一覧を見る
$ ls

# ファイルの中身を見る
$ cat hello.txt

# 新しいファイルを作る（エディタで作るのが普通だが、コマンドでもできる）
$ echo "Hello" > hello.txt
```

`$` はプロンプト（入力待ちの記号）です。自分で入力する必要はありません。

基本コマンド①: 現在地の確認と移動

```
# 今いる場所（ディレクトリ）を表示する
$ pwd
/workspaces/web-app-intro-class-2026

# ディレクトリを移動する
$ cd session01          # session01 フォルダに移動
$ cd ..                 # 1つ上のフォルダに戻る
$ cd ~                  # ホームディレクトリに戻る
```

コマンド	意味	覚え方
`pwd`	現在のディレクトリを表示	**P**rint **W**orking **D**irectory
`cd`	ディレクトリを移動	**C**hange **D**irectory

基本コマンド②: ファイルの一覧と中身の表示

```
# ファイル・フォルダの一覧を表示
$ ls
hello.txt  session01  session02

# 詳細情報付きで表示（サイズ、更新日時など）
$ ls -l

# 隠しファイル（.で始まるファイル）も表示
$ ls -a

# ファイルの中身を表示
$ cat hello.txt
Hello, World! 私の名前は○○です。
```

コマンド	意味	覚え方
`ls`	ファイル一覧を表示	**L**ist
`cat`	ファイルの中身を表示	con**cat**enate（連結）

基本コマンド③: 画面のクリアとコマンドの停止

```
# ターミナルの表示をクリア（きれいにする）
$ clear
# ショートカット: Ctrl + L でも同じ
```

```
# 実行中のコマンドを強制停止する
Ctrl + C
# ping を止め忘れたときなどに使う
```

よくあるトラブル:

- `command not found` → コマンド名のスペルミス。大文字/小文字も区別される
- コマンドが終わらない → `Ctrl + C` で停止

実習3: Codespacesを起動し、hello.txtを作成する

手順

1. Forkした自分のリポジトリをブラウザで開く
2. 緑色の **「Code」** ボタンをクリック
3. **「Codespaces」** タブを選択
4. **「Create codespace on main」** をクリック
5. VS Codeのような画面が表示されるまで待つ（1〜2分）
6. サイドバーの **Explorerアイコン** でファイル一覧を確認

7. エディタエリアで **新しいファイル** を作成: `hello.txt`
    - 内容: `Hello, World! webアプリケーション開発`
8. ファイルを保存: `Ctrl + S` （Mac: `Cmd + S`）
9. ターミナルを開く: メニュー → Terminal → New Terminal
10. 以下のコマンドを実行:

```
cat hello.txt
```

確認ポイント: ターミナルにファイルの内容が表示されればOK

Gitの基本コマンド

ここで、Gitの基本コマンドを4つ覚えましょう:

```
# 1. 状態を確認する（変更されたファイルを見る）
git status

# 2. 変更をステージングする（コミットに含めるファイルを選ぶ）
git add ファイル名

# 3. コミットする（セーブポイントを作る）
git commit -m "コミットメッセージ"

# 4. GitHubにアップロードする
git push
```

流れ:

コミットメッセージには **「何をしたか」** を書きます。

例: `"hello.txtを作成"`

実習4: 最初のコミットとプッシュ

手順

1. Codespacesのターミナルで以下を実行:

```
git status
git add hello.txt
git commit -m "hello.txtを作成"
git log --oneline
```

2. `git log` で自分のコミット履歴が表示されることを確認

3. GitHubにプッシュ（アップロード）する:

```
git push
```

4. ブラウザで自分のリポジトリページを開く
    - `https://github.com/自分のユーザー名/リポジトリ名`
5. ファイル一覧に **hello.txt** が表示されていることを確認
6. **hello.txt** をクリックして内容が正しいことを確認
7. URLをフォームで提出する
    - 例: `httops://github.com/ユーザー名/リポジトリ名/blob/main/hello.txt`

確認ポイント: GitHubのリポジトリページに `hello.txt` が表示されればOK

4. インターネットの仕組み

インターネットとは

インターネット = 世界中のコンピュータをつなぐネットワーク

普段何気なく使っていますが、裏側ではいくつかの重要な仕組みが動いています。

今回は以下の3つを学びます:

1. **IPアドレス** — コンピュータの住所
2. **DNS** — 名前から住所を調べる仕組み
3. **パケット** — データを運ぶ仕組み

IPアドレス — コンピュータの「住所」

IPアドレス = インターネット上のコンピュータを識別する番号

現実世界の住所と同じ:

- 手紙を送るには「住所」が必要
- インターネットで通信するには「IPアドレス」が必要

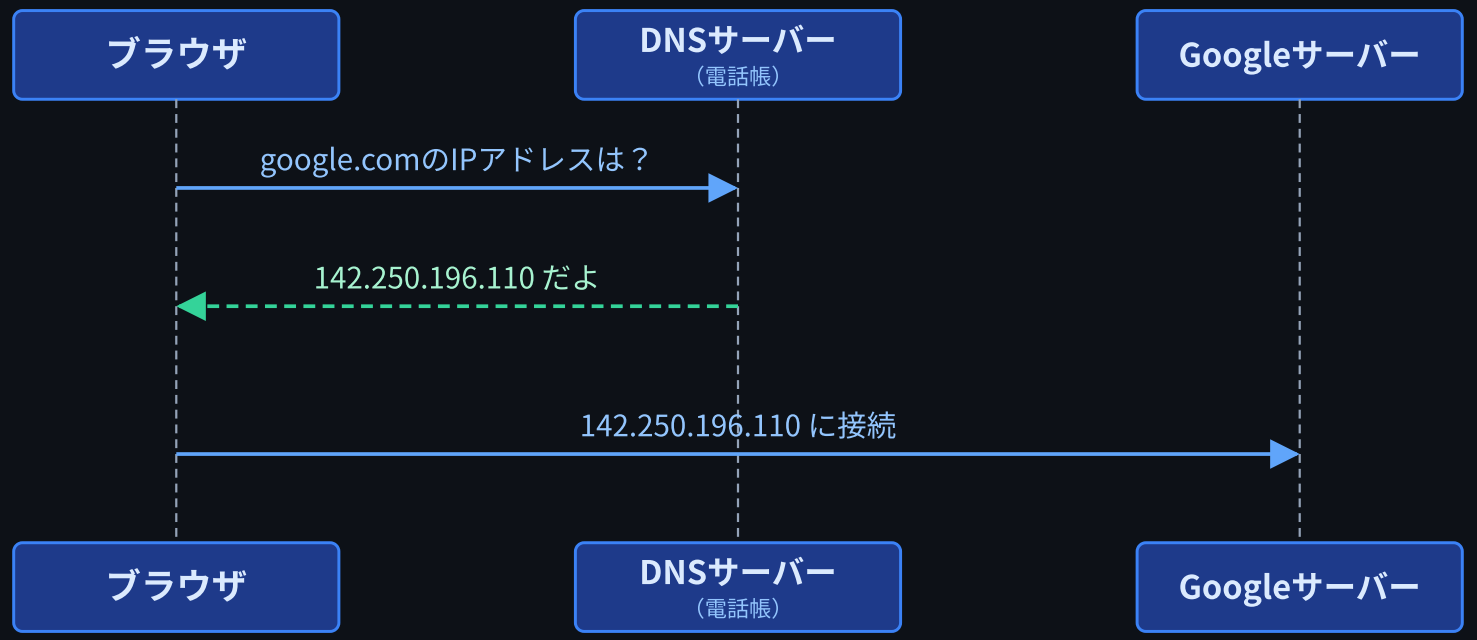
```
例:
  Google のIPアドレス: 142.250.196.110
  自分のPC のIPアドレス: 192.168.1.5（ローカル）

  形式: xxx.xxx.xxx.xxx（0〜255の数字が4つ）
  ※ これはIPv4の場合。IPv6はもっと長い
```

すべてのサーバーが固有のIPアドレスを持っている = 住所があるから手紙（データ）が届く

DNS — インターネットの「電話帳」

DNS（Domain Name System）= ドメイン名をIPアドレスに変換する仕組み



なぜ必要？: `142.250.196.110` を覚えるのは大変。`google.com` なら覚えやすい。

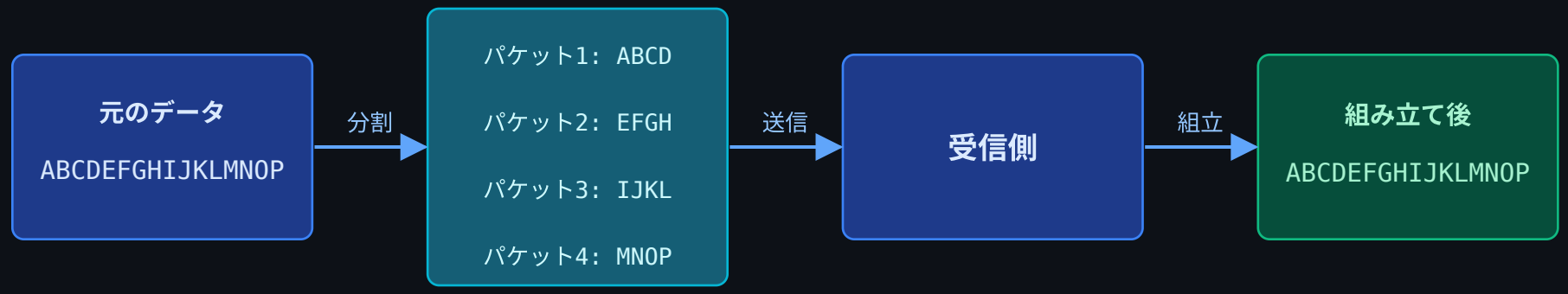
パケット — データを運ぶ「手紙」

パケット = データを小さく分割して送る仕組み

大きなデータ（画像、動画など）をそのまま送ると:

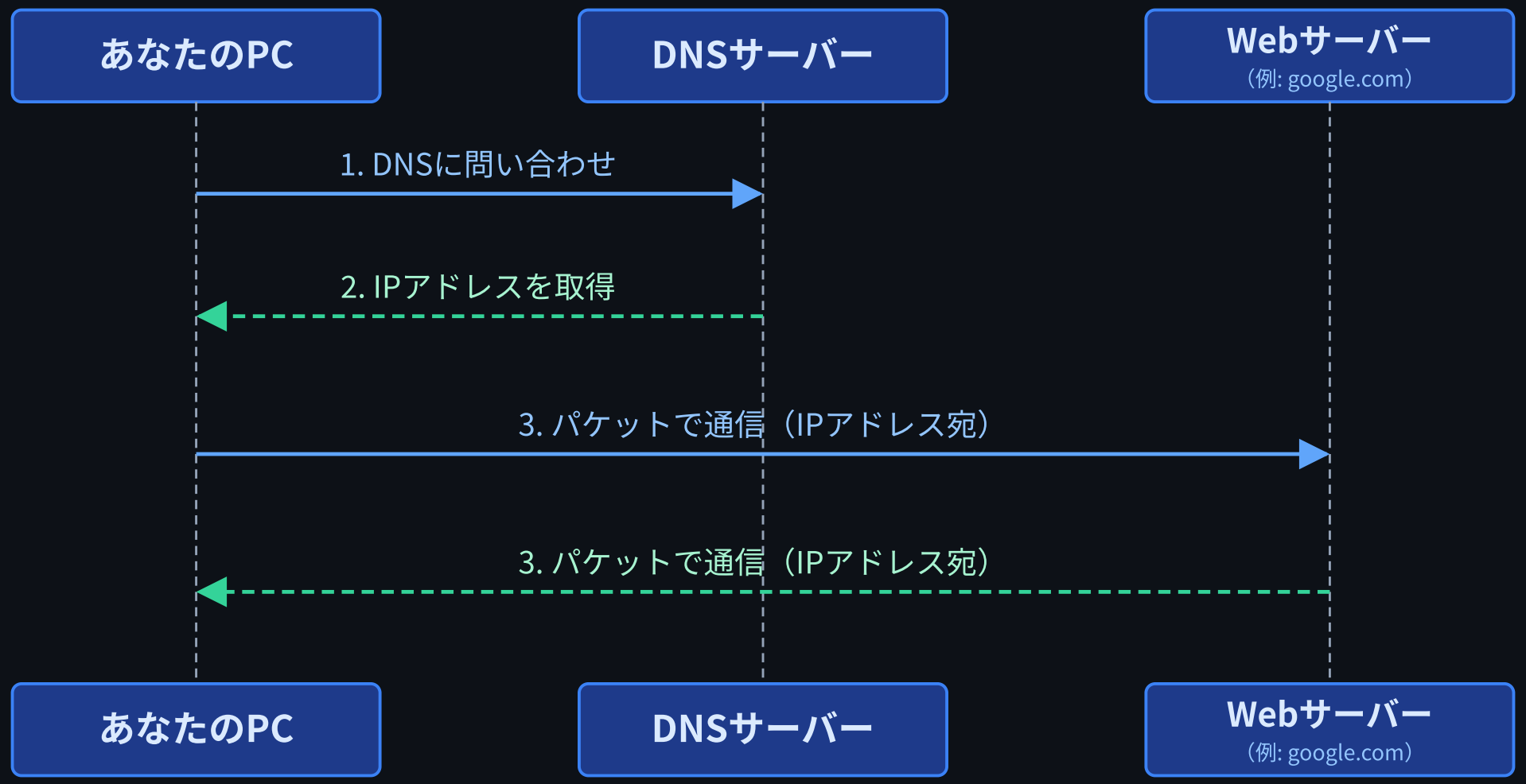
- 回線を長時間占有してしまう
- 途中でエラーが起きたら全部やり直し

そこで、**小さな「パケット」に分割**して送ります:



パケット3だけエラーになっても、パケット3だけ再送すればOK。

まとめ: インターネットの仕組み



実習5: ターミナルでインターネットの仕組みを体感する

手順

1. Codespacesのターミナルを開く
2. `ping` コマンドでサーバーとの通信を確認:

```
ping -c 4 google.com
```

- `-c 4` は4回だけ送信する指定
- `time=` の数値がサーバーまでの往復時間（ミリ秒）

3. `nslookup` コマンドでDNSを確認:

```
nslookup google.com
```

- 「Address:」の行がIPアドレス

4. 別のサイトでも試してみよう:

```
ping -c 4 github.com
nslookup github.com
```

確認ポイント: IPアドレスが表示され、通信時間が確認できればOK

5. クライアントとサーバー

クライアント・サーバーモデル — レストランの比喩

Webの通信は **レストラン** に例えるとわかりやすい:

レストラン	Web	役割
お客さん	**クライアント**（ブラウザ）	注文する側
ウェイター	**HTTP**	注文を運ぶ手段
キッチン	**サーバー**	注文を受けて料理を作る側
料理	**レスポンス**（HTML等）	注文に対する返答
メニュー	**URL**	何を頼むか指定

お客さん（クライアント） が **メニュー（URL）** を見て注文し、
キッチン（サーバー） が料理を作って返す。これがWebの基本。

リクエストとレスポンス

Webの通信は常に **リクエスト（要求）** と **レスポンス（応答）** のペアです。

リクエストの例:

- 「google.com のトップページをください」
- 「TODOリストのデータをください」
- 「新しいTODOを保存してください」

レスポンスの例:

- HTMLファイル（ページの内容）
- JSONデータ（TODOリスト）
- 「保存しました」という結果

HTTPとは

HTTP（HyperText Transfer Protocol）= Webの通信ルール

「Protocol（プロトコル）」= 通信の**約束事・ルール**

例えるなら:

- 日本語で話す = 「日本語」というプロトコル
- 手紙を出す = 「郵便」というプロトコル
- Webで通信する = 「HTTP」というプロトコル

URLの先頭 `https://` はHTTPで通信することを示しています。
（`s` は Secure = 暗号化されている、という意味）

```
https://www.google.com/search?q=hello
  │        │              │       │
  │        │              │       └── パラメータ（検索語）
  │        │              └── パス（何のページか）
  │        └── ドメイン名（どのサーバーか）
  └── プロトコル（通信のルール）
```

URLの構造

URLはWebページの「住所」です。構造を理解しましょう。

```
https://www.example.com:443/products/list?category=books&page=2#section1
  │          │          │       │              │                    │
  │          │          │       │              │                    └ フラグメント
  │          │          │       │              └ クエリパラメータ      (ページ内位置)
  │          │          │       └ パス（どのページか）
  │          │          └ ポート番号（通常は省略）
  │          └ ドメイン名（サーバーの名前）
  └ スキーム（プロトコル）
```

部分	例	意味
スキーム	`https`	通信の方式
ドメイン名	`www.example.com`	サーバーの名前
パス	`/products/list`	サーバー内のどのページか
クエリ	`?category=books`	追加の条件

実習6: 開発者ツールでリクエスト/レスポンスを観察する

手順

1. ブラウザ（Chrome推奨）で新しいタブを開く
2. **開発者ツール** を開く（`F12` / `Ctrl+Shift+I` / `Cmd+Option+I`）
3. **「Network」タブ** を選択
4. アドレスバーに `https://www.google.com` と入力してEnter
5. Networkタブに大量の通信ログが表示される

実習6: 開発者ツールでリクエスト/レスポンスを観察する（続き）

観察ポイント:

- 1つのページ表示で **たくさんの通信** が発生している
- 各行をクリックすると詳細が見える
- **Headers** タブ: リクエストとレスポンスのヘッダー情報
- **Status** 列: `200` = 成功

6. 任意の通信をクリックし、以下を確認:
    - Request URL（どこに送ったか）
    - Request Method（GET / POST）
    - Status Code（200など）

6. HTTPの詳細

HTTPメソッド — 「何をしたいか」を伝える

HTTPリクエストには **メソッド** という種類があります。

メソッド	意味	レストランの例	TODOアプリの例
GET	データを取得	メニューを見る	TODOリストを表示
POST	データを新規作成	料理を注文	新しいTODOを追加
PUT	データを更新	注文を変更	TODOの内容を修正
DELETE	データを削除	注文をキャンセル	TODOを削除

まずは **GET** と **POST** を覚えましょう:

- **GET**: URLにアクセスするだけ。情報をもらう。
- **POST**: フォームの送信など。サーバーにデータを送る。

HTTPステータスコード — 「結果」を伝える

サーバーからのレスポンスには **ステータスコード** が含まれます。

コード	意味	説明
200	OK	成功。正常に処理された
201	Created	作成成功。新しいデータが作られた
301	Moved Permanently	引っ越しました（リダイレクト）
400	Bad Request	リクエストが不正
403	Forbidden	アクセス権限がない
404	Not Found	ページが見つからない
500	Internal Server Error	サーバー内部のエラー

ステータスコードの覚え方

番号の **百の位** で大まかな意味がわかります:

```
2xx ── 成功
3xx ── リダイレクト（転送）
4xx ── クライアントエラー（あなたのせい）
5xx ── サーバーエラー（サーバーのせい）
```

よく出会うもの:

- **200**: ページが正常に表示された
- **404**: URLを間違えた、ページが存在しない
- **500**: サーバー側でプログラムが壊れている

404と500の違い: 404は「その住所には誰もいない」、500は「住人はいるが体調不良」

HTTPリクエスト/レスポンスの中身

リクエスト（ブラウザ → サーバー）：

```
GET /index.html HTTP/1.1
Host: www.example.com
User-Agent: Chrome/120.0
Accept: text/html
```

レスポンス（サーバー → ブラウザ）：

```
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 1234

<html><body><h1>Hello, World!</h1></body></html>
```

- 1行目: メソッド / ステータスコード
- 2行目以降: ヘッダー（メタ情報）
- 空行の後: ボディ（本文）

実習7: 404を体験する

手順

1. ブラウザで存在しないURLにアクセスする:
   - 例: `https://github.com/このページは存在しない12345`
2. **「404」** のエラーページが表示されることを確認
3. 開発者ツールのNetworkタブでステータスコード **404** を確認

確認ポイント: Networkタブで **404** のステータスコードが確認できればOK

7. 困ったときは — トラブルシューティング

まずは自分で試そう

作業中につまずいたら、まず以下を試してみてください。これだけで解決することも多いです。

症状	試すこと
エラーが出た	エラーメッセージをよく読む。**最後の行**に原因が書かれていることが多い
`command not found`	コマンド名のスペルミス。大文字/小文字も区別される
コマンドが終わらない	`Ctrl + C` で停止
ターミナルの表示が崩れた	`clear` または `Ctrl + L` でリセット
Codespaces全体が反応しない	ブラウザのタブをリロード（`F5`）
それでも分からない	エラーメッセージを**そのままコピーして検索**する

それでも解決しないときは、まず **GitHub Copilot Chat** に聞いてみましょう。

GitHub Copilot Chat に聞いてみる

GitHub Copilot Chat = VS Code 内で AI にコードやエラーについて質問できる拡張機能

このCodespacesには **最初から入っています。**

無料版でできること（2026年5月時点）:

- **コード補完: 無制限**
- **チャット: 月 50 メッセージ**
- 使えるモデル: Claude Haiku 4.5

始め方:

1. アクティビティバーの **Copilot アイコン**（吹き出しマーク）をクリック
2. **GitHub アカウントでサインイン**（無料プランが自動で有効になる）
3. チャット欄に質問やエラーメッセージを入力して Enter

> 利用枠は変わることがあります。最新は GitHub Docs - Copilot plans を確認

Copilot Chat — 質問のコツ

うまく質問するほど、的確な答えが返ってきます。

良い質問の例:

```
このコマンドを実行したら以下のエラーが出ました。原因と直し方を教えてください。

実行: pyhton hello.py
エラー: bash: pyhton: command not found
```

含めるとよい情報:

- **エラーメッセージをそのまま貼り付ける**（要約しない）
- **何をしようとしたか**（実行したコマンドや操作）
- **期待した結果と実際の結果**

便利な使い方:

- ターミナルのエラー行を選択 → 右クリック → **「Explain with Copilot」** で解説してもらえる
- エディタでコードを選択して `Ctrl + I`（Mac: `Cmd + I`）でその場で質問できる

VS Code Live Share でヘルプを求める

VS Code Live Share = 自分のCodespacesの画面を**リアルタイムで他の人と共有**できる拡張機能

ヘルプを求めるときに使えること:

- 講師が**自分のコードをそのまま**見られる
- 講師が**コードを直接編集**して直してくれる（許可した場合）
- **ターミナル**も共有できる（エラーの再現をその場で見せられる）

口頭やスクリーンショットで説明するより、はるかに早く解決します。

Live Share — 手順① インストールとサインイン

1. アクティビティバー（左端の縦のメニュー）の **拡張機能アイコン**（四角が4つ）をクリック
2. 検索ボックスに `Live Share` と入力
3. Microsoft 提供の **Live Share** をインストール
4. インストール後、アクティビティバーに追加された **Live Share アイコン**（矢印マーク）をクリック
5. 「Sign in」を選び、**GitHub アカウント**でサインイン

> Codespacesには最初から入っている場合もあります。アクティビティバーに Live Share アイコンが既にあればインストールは不要です。

Live Share — 手順② 招待リンクを講師に送る

1. アクティビティバーの **Live Share** タブを開く
2. **「Share (Read/Write)」** をクリック
   - 講師がその場でコードを直接編集して直せるモード
3. 「Invitation link copied to clipboard!」という通知が表示される
4. **招待リンクが自動でクリップボードにコピー**される
5. **講師にチャット等でリンクを貼り付けて送信**する

参加してもらえると、講師の名前が Live Share タブに表示され、エディタ上にカーソルが見えるようになります。

Live Share — 手順③ 注意点と終了方法

セキュリティ上の注意:

- 招待リンクを知っている人は誰でも参加できます。**講師以外には絶対に送らない**
- リンクは**SNSや公開チャンネルに貼らない**

解決したら必ずセッションを終了する:

1. アクティビティバーの **Live Share** タブを開く
2. 上部の **「Stop Collaboration Session」**（停止ボタン）をクリック
3. 「Session has ended.」と表示されれば終了完了

本日のまとめ

学んだこと

1. **Webアプリ** = ブラウザで動くアプリ。フロントエンドとバックエンドがある
2. **Git/GitHub** = バージョン管理。セーブポイントを作れる
3. **Codespaces** = ブラウザで使える開発環境
4. **インターネット** = IPアドレス（住所）+ DNS（電話帳）+ パケット（手紙）
5. **クライアント/サーバー** = 注文する側と作る側。HTTPで通信
6. **HTTP** = メソッド（GET/POST）とステータスコード（200/404/500）

次回予告

第2回: HTML基礎 — TODOアプリの骨格を作る

HTMLタグを学び、TODOアプリの画面を作り始めます。

提出物

実習で作成したファイルをフォームから提出してください:

1. GitHubリポジトリのURL
   - 例: `https://github.com/ユーザー名/リポジトリ名`
2. `hello.txt` のGitHubのURL
   - 例: `https://github.com/ユーザー名/リポジトリ名/blob/main/hello.txt`

お疲れ様でした！